StagedVulBERT: Multi-Granular Vulnerability Detection with a Novel Pre-trained Code Model

Yuan Jiang, Yujian Zhang, Xiaohong Su, Christoph Treude and Tiantian Wang

Abstract—The emergence of pre-trained model-based vulnerability detection methods has significantly advanced the field of automated vulnerability detection. However, these methods still face several challenges, such as difficulty in learning effective feature representations of statements for fine-grained predictions and struggling to process overly long code sequences. To address these issues, this study introduces StagedVulBERT, a novel vulnerability detection framework that leverages a pre-trained code language model and employs a coarse-to-fine strategy. The key innovation and contribution of our research lies in the development of the CodeBERT-HLS component within our framework, specialized in hierarchical, layered, and semantic encoding. This component is designed to capture semantics at both the token and statement levels simultaneously, which is crucial for achieving more accurate multi-granular vulnerability detection. Additionally, CodeBERT-HLS efficiently processes longer code token sequences, making it more suited to real-world vulnerability detection. Comprehensive experiments demonstrate that our method enhances the performance of vulnerability detection at both coarse- and fine-grained levels. Specifically, in coarse-grained vulnerability detection, StagedVulBERT achieves an F1 score of 92.26%, marking a 6.58% improvement over the best-performing methods. At the fine-grained level, our method achieves a Top-5% accuracy of 65.69%, which outperforms the state-of-the-art methods by up to 75.17%.

Index Terms—Vulnerability detection, Code language model, Pre-training task, Program representation

arXiv:2410.05766v1 [cs.CR] 8 Oct 2024

1 Introduction

Software vulnerability discovery is a classic problem that becomes challenging due to its undecidable nature in the general case [1], [2]. Recently, with the further development of deep learning (DL) technologies, data-driven vulnerability detection with automatic extraction of vulnerability patterns has attracted many researchers to invest in this prominent research field. These approaches [3] are often formulated as a classification task, which takes code features as input, mapping them to the binarization of the probability distribution that indicates whether or not the program contains a vulnerability. Although these methods (e.g., Devign [4], Reveal [5], IVDetect [6], AMPLE [7], VulDeePecker [8] and SySeVR [9]) have improved vulnerability detection by considering complex structural information and various code representations, their effectiveness remains constrained. The primary reason is that models trained solely on limited vulnerability datasets face difficulties in extracting complex semantic patterns, leading to inadequate performance and restricted practicality in detecting software vulnerabilities in real-world scenarios.

To address such shortcomings, the state-of-the-art methods LineVul [10], VulBERTa [11] and SVulD [12] adopt a new learning paradigm of "pre-training and fine-tuning" with the Pre-trained Code Language (PCL) models for source code representation and vulnerability detection. Concretely, these approachess, different from other DL-based detection methods, involve self-supervised pre-training on large-scale open-source code datasets, followed by fine-tuning on vulnerability datasets, thereby achieving superior performance compared to traditional models. A recent empirical study also demonstrates that the transformer-based method LineVul achieves the best performance among the 11 cutting-edge DL-based detection methods [13]. In addition, among transformer-based methods, LineVul is the only one designed for multi-granular vulnerability detection, while the others (e.g., SVulD and VulBERTa) are limited to coarse-grained vulnerability detection.

Although current efforts made by pre-trained model-based approaches (e.g., LineVul) have improved the detection capabilities, they still exhibit the following weaknesses: (1) they mainly treat code as a sequence of tokens and adopt the basic transformer architecture to capture the relationships between tokens. However, relying solely on token-level features makes it difficult to achieve excellent performance in statement-level vulnerability detection, as the cases provided in Section 2.1 show; (2) they, if designed for multi-granular vulnerability, solely employ function-level (coarse-grained) labels for training models to achieve fine-grained detection in unseen programs. The objective gaps between training and prediction [14] may lead to poor performance in pinpointing vulnerable lines; (3) they predominantly build on the existing PCL models (e.g., CodeBERT [10], [15] or UniXcoder [12]), which are limited by sequence lengths – typically a maximum of 512 tokens. Thus, code functions that exceed the maximum length (i.e., 512) will simply be truncated, which may result in the loss of useful information.

To overcome the limitations of the state-of-the-art method, in this study, we propose *StagedVulBERT*, *a staged vulnerability detection framework employing CodeBERT-HLS*,

- *Y. Jiang (jiangyuan@hit.edu.cn), Y. Zhang (23S003081@stu.hit.edu.cn), X. Su (sxh@hit.edu.cn), and T. Wang (wangtiantian@hit.edu.cn), are with the School of Computer Science and Technology, Harbin Institute of Technology, Harbin, Heilongjiang, 15001.*
- *C. Treude is with the School of Computing and Information Systems, Singapore Management University, Singapore. E-mail: ctreude@smu.edu.sg*

which offers several key advantages. Firstly, it leverages the proposed CodeBERT-HLS to integrate low-level (i.e., token-level) features and high-level (i.e., statement-level) semantic features to accurately represent programs. Secondly, it directly uses coarse- and fine-grained labels to construct distinct binary classifiers through supervised learning for coarse- and fine-grained vulnerability detection. Thirdly, it is equipped with a novel algorithm that can segment the token sequence and then effectively apply various components of CodeBERT-HLS to process longer token sequences. The three innovations respectively address the aforementioned limitations of previous transformer-based works. In StagedVulBERT, CodeBERT-HLS serves as the core component, representing an advanced pre-trained model specialized in **H**ierarchical **L**ayered **S**emantic encoding for source code. A key innovation in CodeBERT-HLS is that it employs an architecture featuring segmented Transformer encoders. This design is crucial for simultaneously capturing semantics at both token and statement levels and for processing longer token sequences efficiently. Additionally, a novel pre-training strategy, the Masked Statement Prediction (MSP) model, has been developed for CodeBERT-HLS to enhance pre-training effectiveness. Consequently, on real-world vulnerability datasets, our approach has been proven to more accurately detect vulnerabilities compared to existing methods through extensive evaluations.

The main **contributions** of our paper are:

- **New vulnerability detection framework**. We construct a novel vulnerability detection framework, StagedVulBERT, which not only processes longer token sequences than existing models but also performs the vulnerability detection task with high performance at both the coarse- and fine-grained levels.
- **New pre-trained code language model for vulnerability detection**. We introduce a new pre-trained code language model CodeBERT-HLS specifically designed for multi-granular vulnerability detection, which has the advantage of modeling code semantics at different levels of granularity.
- **New pre-training task for code representation model**. We develop a novel pre-training task tailored to enhance the model's comprehension of code syntax and semantic relationships, which is crucial for achieving high accuracy in the downstream vulnerability detection task.

2 Background and Motivation

2.1 Motivating Example

Fig. 1 depicts a vulnerability (CVE-2019-13304) in ImageMagick, which is classified as CWE-787 in the National Vulnerability Database. This example shows a stack-based buffer overflow due to improper buffer size checks in lines 197-207. The code declares an array for image pixels (line 184) and erroneously allows overflow when writing newline characters (line 204, marked in red dashed boxes), leading to memory corruption. When applying the state-of-the-art method LineVul for vulnerability localization on specific segments of the program (lines 184-211), the results are displayed as a heatmap, which is illustrated on the right

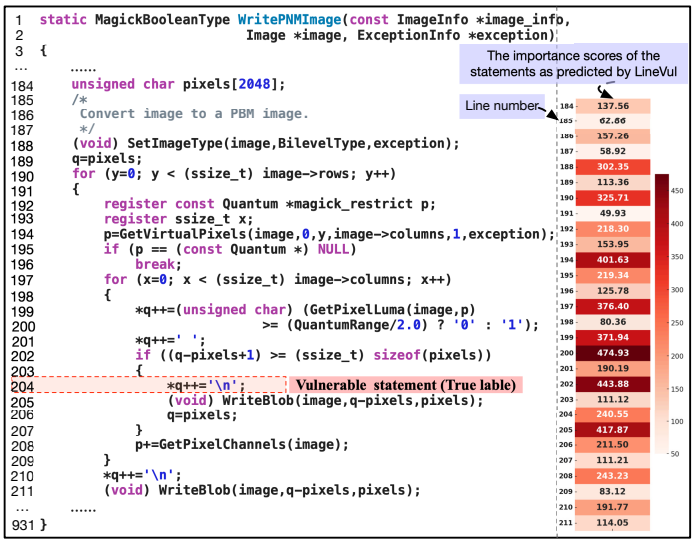


Fig. 1. A real-world example (i.e., CVE-2019-13304) of stack-based buffer overflow, which occurs from writing excess data to an array due to improper buffer size check.

side of Fig. 1. In this heatmap, each element represents the importance score assigned by LineVul to each line of code, indicating its contribution to the vulnerability detection result.

From the example, we can make three observations. The first is that vulnerabilities in real-world projects (e.g., Fig. 1) are often complex to understand and may require considerable expertise for detection. However, existing DL-based methods were mainly trained on vulnerability datasets (e.g., big_vul with approximately 180,000 samples), which are relatively small compared to other available general code databases (e.g., CodeSearchNet with approximately 8,000,000 samples) due to the high cost of labeling. Recent works [10], [11] have demonstrated that first pre-training code language models on large-scale code datasets like CodeSearchNet can improve the semantic understanding of vulnerable code, and then fine-tuning these models on vulnerability datasets can enhance their detection capabilities. However, most of these methods are based on the original transformer architecture, which does not perform well for multi-granular vulnerability detection. Therefore, in this study, we develop a new PCL model, CodeBERT-HLS, tailored for multi-granular vulnerability detection, and design suitable pre-training tasks.

The second observation is that some vulnerable programs can be very long, spanning thousands of tokens. Therefore, most DL-based methods struggle to detect such vulnerabilities due to a limited input token number. For example, the best-performing transformer-based method, LineVul, for vulnerability detection only accepts up to 512 tokens [10], which cannot detect the vulnerable example in Fig. 1 because the vulnerable statement exceeds the first 512 tokens. Although recently proposed Large Language Models (LLMs) seem to address the challenge of limited length, they perform worse than transformer-based methods [16], [17]. Therefore, in this study, we develop an advanced algorithm tailored to the proposed transformer-based method for effective, long-sequence vulnerability detection.

The third observation is that most transformer-based methods for vulnerability detection perform poorly when applied to the fine-grained level. As the heatmap in Fig. 1 demonstrates, in the code segment between lines 184-211, the most likely vulnerability predicted by LineVul is at Line 200, rather than the actual vulnerable statement at Line 204. This indicates that the state-of-the-art method's fine-grained vulnerability prediction is not sufficiently accurate. The reasons for the failed detection are: (1) LineVul trains CodeBERT only at a coarse level, causing misalignment with fine-grained detection goals [14]; (2) LineVul generates features at the token and program levels, which are not tailored for statement-level vulnerability detection. Fine-grained labeling marks specific vulnerable lines within functions, and all statements corresponding to fine-grained labels form the vulnerability-relevant structure of the code snippet. Therefore, employing fine-grained labels in model training can reduce the above training-predicting gap and provide more precise signs to guide the model in learning about the specific code statements causing vulnerabilities. To achieve this, we propose CodeBERT-HLS to generate statement representations, which, along with fine-grained labels, help construct a more accurate fine-grained detection model.

We emphasize once more that although recent LLMs have addressed many challenges of DL- or transformer-based methods mentioned in the above three observations, empirical studies [16], [17] suggest they do not outperform specific transformer-based methods like LineVul in vulnerability detection, even post-fine-tuning. Hence, given that current LLMs are suboptimal in detecting vulnerabilities, improving transformer-based methods to address these challenges is a valuable research direction.

2.2 Background

Since this study aims to enhance transformer-based methods, a category of DL-based methods, we begin with an overview of DL-based methods before delving into the specifics of transformers.

2.2.1 DL-based Vulnerability Detection Methods

Due to programs typically being written by humans, they share many important properties with natural language text (e.g., repetitive, predictable, and long-term dependencies) [18], [19]. Thus, more and more researchers focus on capturing program features by means of statistical language models. Numerous studies also demonstrate that neural networks achieve superior results with source code, similar to their performance with natural language [20], [21]. Moreover, a DL model, such as LSTM, itself is a good feature extractor, hence, it can extract features in an automatic way instead of designing various sophisticated feature extraction rules [22]. However, most DL-based models struggle to effectively learn vulnerability patterns for both coarse- and fine-grained detection. Therefore, further research into advanced network architectures suited for multi-granular vulnerability detection is necessary. To make the concept of "multi-granular vulnerability detection" more clear, we give its definition as follows: Multi-granular vulnerability detection involves identifying and classifying vulnerabilities at various levels of code abstraction, such as function and statement levels, using techniques like static analysis, dynamic analysis, or data-driven methods.

2.2.2 Transformer-based Vulnerability Detection Methods

Inspired by the significant success of pre-training in NLP, researchers have begun applying transformers to vulnerability detection with extensive empirical evidence demonstrating their effectiveness over traditional DL-based methods [13]. In this study, following previous work [16], we categorize transformer-based methods used for vulnerability detection into two groups: LLM-based and specific transformer-based. LLM-based methods, well-trained with billions of parameters and training samples, generally perform well in various software engineering tasks that require high language comprehension and generation capabilities. The LLMs recently proposed and utilized extensively include DeepSeek-Coder [23], CodeLlama [24], StarCoder [25], WizardCoder [26], Mistral [27], Phi-2 [28], and ChatGPT [29].

Specific transformer-based approaches have emerged to utilize PCL models for detecting vulnerabilities. These methods also adhere to strategies that first involve pre-training, followed by fine-tuning. Notable PCL-based vulnerability detection methods include LineVul, VulBERTa and SVulD. Most of these methods are based on the basic transformer architecture and well-known pre-training tasks (e.g., Masked Language Modeling (MLM) [30], ULM [31], and Contrastive Learning (CL) [32]. Note that pre-training tasks refer to specific training objectives designed to help the model learn general representations from large, unlabelled datasets via self-supervised learning.

In this study, we focus on enhancing specific transformer-based methods, which empirical evidence indicates as superior to other LLM-based approaches [16], [17], yet they exhibit the following limitations: (1) most PCL models in transformer-based methods target coarse-grained vulnerability detection, with fewer addressing detection at multiple granularities; (2) existing PCL models typically generate feature representations at either the token or program level, which challenges statement-level vulnerability detection due to the lack of specific statement features; and (3) most pre-training tasks are designed for the standard transformer architecture and may not be suitable when this architecture is modified. Therefore, this paper develops a novel PCL model and an associated pre-training task, aiming to enhance multi-granular vulnerability detection to address these limitations.

3 The Overall Framework of Our Method

To address the challenge of accurately extracting vulnerability semantics from small datasets, we adopt a strategy of first pre-training on large-scale open-source code datasets, followed by fine-tuning on a relatively small vulnerability dataset, a method previously proven to enhance detection accuracy effectively [10]. Our approach, distinct from existing pre-training-based methods [10], [11], introduces a novel transformer-based model architecture CodeBERT-HLS and a pre-training task MSP, which are particularly suitable for multi-granular vulnerability detection. The work process of our model architecture, CodeBERT-HLS, is illustrated at the top of Fig. 2, while the workflow of pre-training and fine-tuning CodeBERT-HLS is shown at the bottom of Fig. 2.

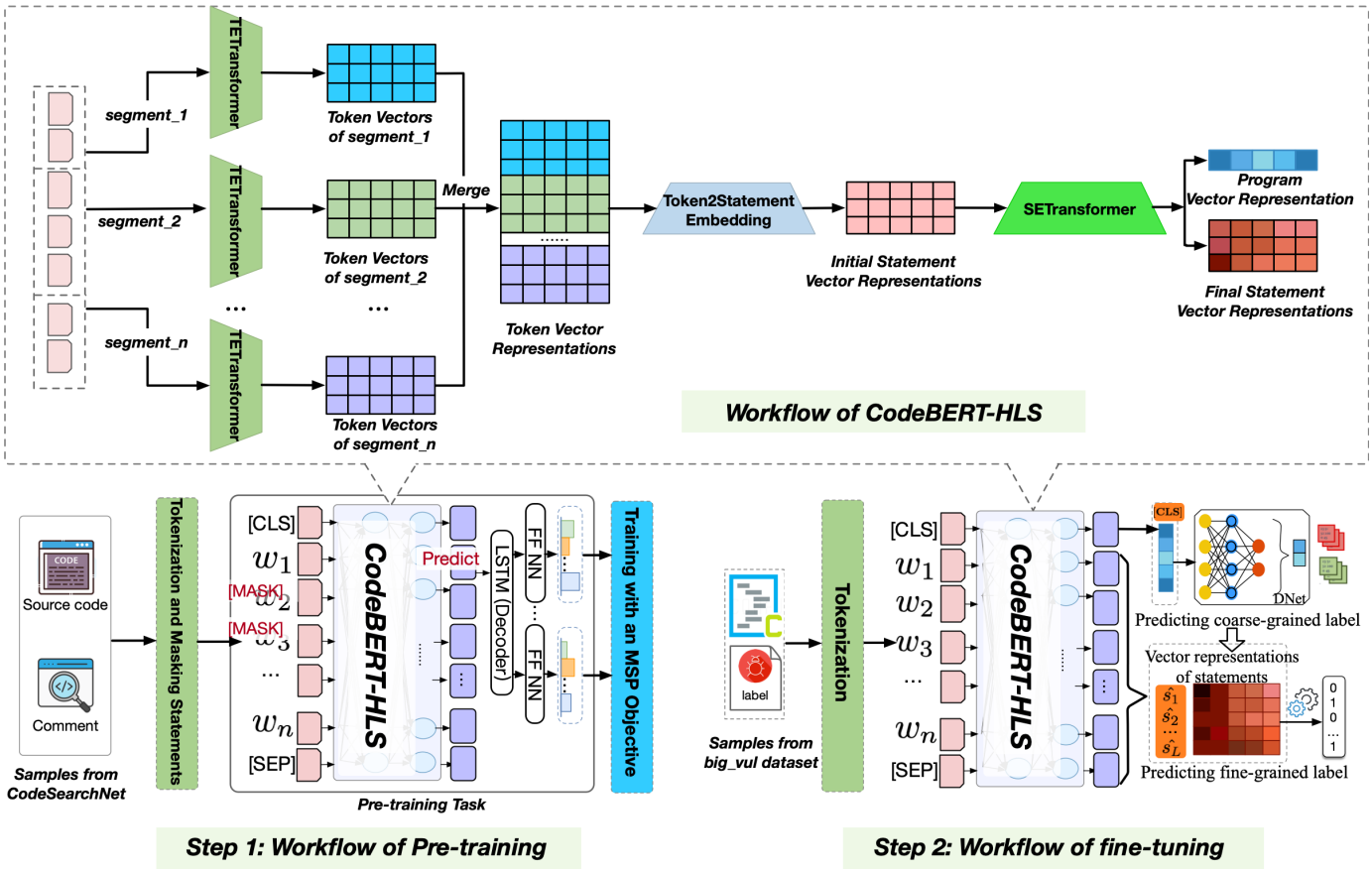


Fig. 2. Workflow of the proposed StagedVulBERT, which includes two steps: pre-training and fine-tuning. The Workflow of CodeBERT-HLS is illustrated at the top of the figure.

3.1 Workflow of CodeBERT-HLS

Unlike most related work [10], [11] that uses existing PCL models (e.g., CodeBERT) capturing only token-level information from token sequences, our approach involves exploring staged code representation with the proposed PCL model CodeBERT-HLS to capture information at both the token-level and statement-level, thereby facilitating a deeper understanding of vulnerable programs. The architecture of CodeBERT-HLS is depicted at the top of Fig. 3, and its workflow is as follows: First, the code snippet is treated as a sequence of code tokens, which are then split into n segments. The Token Encode Transformer (TETransformer) is used to learn features for each segment, generating token vector representations. Next, all token vectors from different segments are merged into a matrix. This matrix is then passed as input to the Token2Statement embedding layer to obtain the initial vector representation of each statement. Finally, CodeBERT-HLS leverages the Statement Encode Transformer (SETransformer) to capture dependency relationships among statements, which allows for the generation of more accurate statement vectors. The reason for splitting token sequences into multiple segments is to effectively handle long code token sequences. More details can be found in Section 4.4.

3.2 Workflow of Pre-training and Fine-tuning

A pre-training process is employed in this study to learn a general-purpose code representation. However, existing pre-training tasks, such as MLM, are only suitable for training models that generate token-level representations. This makes them unsuitable for training CodeBERT-HLS, which is designed to output statement-level representations. Therefore, we design a new pre-training task MSP to align with the architecture of CodeBERT-HLS. The workflow involves three steps: (1) masking tokens from the selected statements in a code sequence and inputting the masked sequence into CodeBERT-HLS, (2) generating a context-based representation for each masked statement, and (3) predicting the tokens of the masked statements based on the statement's representation and calculating the loss to adjust the network parameters.

After pre-training, we fine-tune the pre-trained CodeBERT-HLS model on the big_vul dataset to get suitable weights for our vulnerability detection task. The workflow can be described as follows: (1) inputting a code snippet into CodeBERT-HLS to obtain representations for each statement and the entire code snippet, which can be regarded as feature vectors for vulnerability detection. (2) employing an FFNN named DNet that takes the feature vector of a code snippet as input and predicts the corresponding vulnerability label at the coarse-grained level. (3) once a code snippet is predicted as vulnerable, constructing a fine-grained network that takes a statement vector produced by the SETransformer as input and outputs a probability distribution over the label space $\mathcal{L}$. (4) calculating the cross-entropy losses based on the coarse and fine prediction

results to fine-tune the network parameters.

3.3 Advantages Over Other Methods

Compared to state-of-the-art methods based on PCL models (e.g., LineVul), our proposed approach offers several distinct advantages. Firstly, our method is capable of generating feature representations at both the program and statement levels, making it suitable for downstream multi-granular vulnerability detection. Secondly, it employs fine-grained labels to construct a binary classifier based on statement feature representations. This enables the automatic and more accurate determination of each statement's relevance to vulnerabilities. Thirdly, our method is capable of efficiently processing long token sequences, which enables it to extract more semantic information from the program and achieve better generalization in real-world scenarios. Fourthly, we introduce a pre-training task that is more suitable for our model architecture than the existing tasks (e.g., MLM) are, to enhance our model's understanding of general code semantics through self-supervised learning.

4 Our Approach

This section focuses on introducing the core components of CodeBERT-HLS and the pre-training task tailored specifically for CodeBERT-HLS. Additionally, this section explains how CodeBERT-HLS handles long code token sequences.

4.1 Token Encode Transformer (TETransformer)

TETransformer is designed to capture the semantic relationships between tokens in a program, generating a vector representation for each token in a code fragment. The TETransformer adopts an architecture identical to that of BERT [30] and RoBERTa [33], employing a multi-headed self-attention mechanism. This mechanism allows for simultaneous processing of multiple attention heads, each focusing on different aspects of the input token relationships. As a result, it enables the model to capture diverse contexts and relationships within the input token sequence. This enhanced capability allows for extracting more accurate semantics of vulnerable programs, significantly improving the model's overall performance in detecting vulnerabilities. A formal description of the TETransformer model is as follows:

$$G^n = LN\left(\text{MultiAttn}\ \left(H^{n-1}\right) + H^{n-1}\right) \tag{1}$$

$$H^n = LN\left(FFN\left(G^n\right) + G^n\right) \tag{2}$$

where *MultiAttn* denotes the multi-headed self-attention mechanism, *FFN* represents a two-layer FFNN, and *LN* signifies layer normalization operations. For the nth Transformer layer, the output of the multi-headed self-attention, $\hat{G}^n$, is computed as follows:

$$Q_i = H^{n-1}W_i^Q, K_i = H^{n-1}W_i^K, V_i = H^{n-1}W_i^V \tag{3}$$

$$head_i = \text{softmax}\left(\frac{\mathrm{Q_i K_i^T}}{\sqrt{\mathrm{d_k}}}\right)\mathrm{V_i} \tag{4}$$

$$\hat{G}^n = \left[\text{head}_1; \ldots; \text{head}_u\right] W_n^O \tag{5}$$

where the output of the previous layer, $H^{n-1} \in \mathbb{R}^{|X| \times d_h}$, is linearly projected using learnable model parameter matrices $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_h \times d_k}$ into a triplet of queries Q, keys K, and values V. Here, u denotes the number of heads in the multi-headed mechanism, d_k represents the dimensionality of the feature vector output by each head, and $W_n^O \in \mathbb{R}^{ud_k \times d_h}$ is a learnable model parameter. The parameter W_n^O is used to linearly map the concatenated output of the multi-heads into an output vector with the same dimensionality as that of the input to the Transformer layer.

4.2 Token2Statement Embedding

The obtained sequence of token vectors is fed into the Token2Statement layer to calculate the initial representation for each statement. In this section, we provide the following three strategies used for Token2Statement Embedding, with the most effective strategy to be identified via experimental evaluation.

(1) The first strategy, denoted as 'Average Token2Statement', calculates the average value of the embedding of tokens within a statement. The specific process, illustrated in Fig. 4, is as follows: First, we construct a correspondence matrix $C \in \mathbb{R}^{L \times n}$ between statements and tokens. In this matrix, each row sequentially corresponds to a statement in the program, and each column corresponds to a specific token element in the token sequence. If the token element corresponding to a column is contained within the statement of a row, that position in the matrix is assigned the value 1; otherwise, it is 0. Next, we vertically concatenate all token vectors output by the TETransformer to form the Token matrix $T \in \mathbb{R}^{n \times d_h}$, where each row represents the vector representation of a token in the code sequence. Finally, by multiplying matrix C with matrix T, we obtain the initial vector representation of each statement.

The process of initializing the aforementioned statement vectors essentially involves averaging the vector representations of all tokens contained in a statement. For instance, if a statement c_i includes tokens from w_o to w_p, its initial vector representation $\boldsymbol{s_i}$ can be computed using the following formula.

$$\boldsymbol{s_i} = \frac{1}{|p-o|} \sum_{j \in [o \sim p]} \boldsymbol{w_j} \tag{6}$$

(2) The second strategy, referred to as 'Weighted Average Token2Statement', involves computing the weighted average value of the embeddings of all tokens within a statement. As indicated in formula 6, the Average Token2Statement approach does not account for the varying importance of different tokens within a statement. For instance, API function calls in *CallExpression* statements may carry more vulnerability information and thus hold greater importance [34]. Recognizing this, we propose that when generating the vector representation of a statement from token vectors, the differing impacts of individual tokens should be considered. To achieve this, we introduce a learnable token weight matrix that adaptively fuses the semantic information of different tokens, thereby producing a more accurate initial vector representation for each statement. The

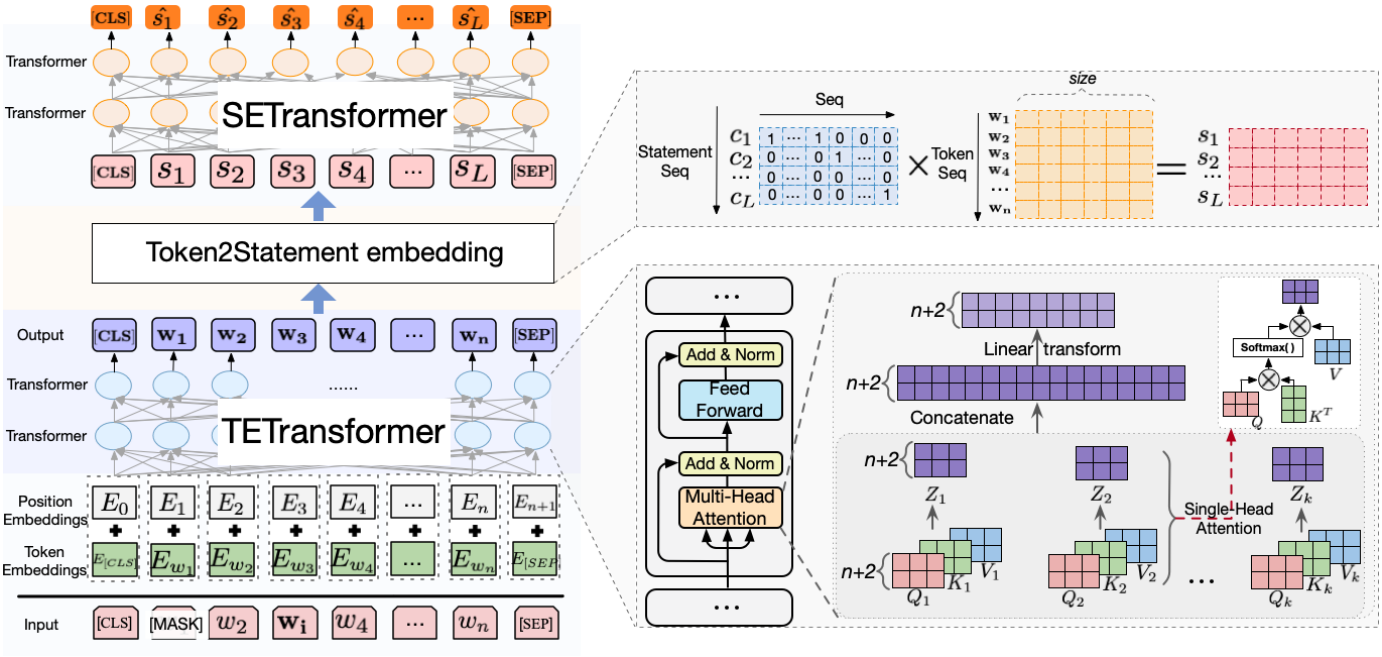


Fig. 3. The overall framework of the CodeBERT-HLS method, which is composed of the following three components: TETransformer, Token2Statement embedding, and SETransformer.

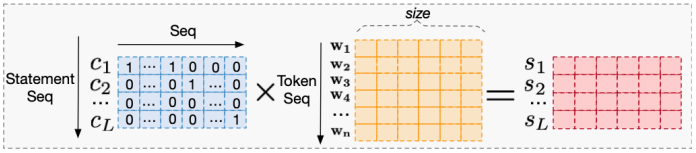


Fig. 4. Initialization of statement vector representations based on token vectors and statement-token correspondence

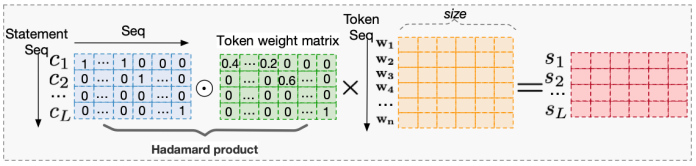


Fig. 5. Initializing statement vector representations using token vectors and statement-token correspondence, while accounting for the varied importance of tokens within statements

process of Token2Statement embedding with the introduction of a weight matrix is illustrated in Fig. 5.

(3) The third strategy, known as 'Attention Average Token2Statement', differs from 'Weighted Average Token2Statement' in that it focuses on the contribution of each token to the representation of the entire code snippet, rather than solely relying on learnable token weights. In 'Attention Average Token2Statement', the weight assigned to each token is determined by its attention score relative to the [CLS] token, which is typically placed at the beginning of the token sequence to encapsulate the semantic information of the entire input code fragment. Specifically, this method first involves using a parameter matrix $\mathbf{Q}$ to map the [CLS] token vector to a query vector q, and a parameter matrix $\mathbf{K}$ to map each token vector to a key vector k. Then, the attention scores are computed as the product of q and k. Finally, these attention scores are applied as weights to calculate the initial vector representation of each statement, which is achieved by performing a weighted summation of all token vectors contained within the statement.

4.3 Statement Encode Transformer (SETransformer)

The SETransformer aims to capture the global semantic features between statements in a program, generating accurate vector representations for each line of code as well as for the entire code snippet. Similar to TETransformer, the SETransformer also employs a multi-headed self-attention mechanism to compute in parallel the degree of relevance between each statement and other dependent statements in the same code sequence. By doing so, it effectively establishes the complex semantic relationships between different statements, thereby achieving a comprehensive understanding and modeling of the program's global semantics.

4.4 Algorithm for Processing Long Code Sequences

In most pre-training settings for code language models, the maximum input sequence length is set to 512 tokens. However, this limitation prevents PCL models from scaling to process long and complex programs with vulnerabilities. To effectively tackle this challenge, we propose a long code encoding algorithm using CodeBERT-HLS, detailed in Algorithm 1. This algorithm is specifically designed to transform long code sequences into vector representations while preserving their semantic information.

Specifically, we assess whether the length of the input code sample exceeds the predefined maximum token sequence length, denoted as *m_len*. If it does, we truncate the sample (Lines 2-3). Then, we calculate the number of segments (i.e., segment_count) required for splitting (Line

Algorithm 1: CodeBERT-HLS encodes long token sequence

Input : Batch of samples, Maximum token sequence length *m_len* (e.g., 512, 1024, 2048)
Output: Vector representations of samples

1 **for** *sample in Batch* **do**
2 **if** *length(sample)* $>$ *m_len* **then**
3 sample = sample[:m_len] //Truncate sample to max length;
4 segment_count = $\left\lceil \frac{length(sample)}{512} \right\rceil$;
5 Split sample into segment_count segments;
6 Initialize an empty list for token vectors: all_token_vectors = [];
7 **for** *segment in segments* **do**
8 token_vectors = **TETransformer**(segment);
9 Append token_vectors to all_token_vectors;
10 merged_token_vectors = Merge(all_token_vectors);
11 state_initial_vectors = **Token2Statement**(merged_token_vectors);
12 state_vectors = **SETransformer**(state_initial_vectors);
13 Output vector representations of each statement and the entire program;

4). Subsequently, the sample is divided into segment_count segments, and each segment is processed through the TETransformer to obtain the token vector representations (Lines 5-9). Next, we concatenate the token vectors from all segments and use Token2Statement to generate the initial vector representation for each line of code (Lines 10-11). Finally, these initial statement vectors are fed into the SETransformer to obtain the final statement vector representation (Lines 12). In this algorithm, the process of splitting each sample and handling each segment dynamically depends on the length of the sample and the parameter for maximum length. Therefore, this approach offers a more flexible way to handle code samples of varying lengths and effectively preserves the semantic information of the original samples.

Although Algorithm 1 processes individual samples for ease of understanding, in our implementation, we accelerate the generation of token vector representations for all samples in a batch by inputting their split code segments collectively into the TETransformer. Then, in the Merge function, segments belonging to the same sample are consolidated to form a matrix of initial token vectors. Finally, these token matrices for all samples in the batch are fed into Token2Statement and SETransformer to obtain vector representations of the statements.

4.5 Pre-Training CodeBERT-HLS

This section focuses on developing a pre-training task to devise more effective feature representations. This foundational work is essential for preparing the model to further specialize in accurately understanding and identifying code vulnerabilities through fine-tuning on labeled datasets. In this paper, we introduce the Masked Statement Prediction (MSP) model, a novel approach designed for effectively pre-training CodeBERT-HLS. Specifically, we select 15% of the code lines from the input code fragment and apply the following transformations: in 80% of these selected lines,

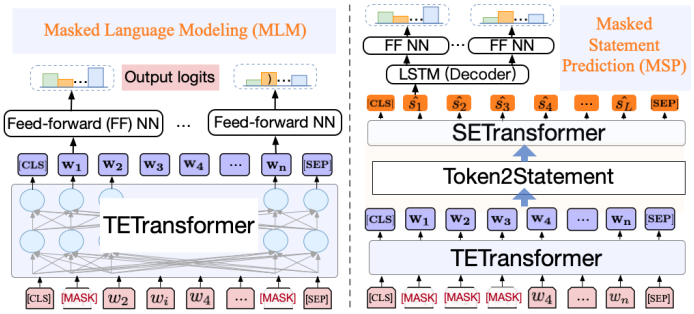


Fig. 6. Comparing the proposed MSP with the existing pre-training task MLM

we replace all tokens with an equal number of [MASK] tokens; in 10% of the lines, we replace the tokens with random token sequences; and we leave the remaining 10% of the lines unchanged. These numbers are chosen to align with the previous work, which commonly employ MLM for pretraining existing PCL models [15], [35]. The objective of the MSP is to predict the original token sequences in these sampled code lines.

$$\mathcal{L}_{MSP}(\theta) = -\sum_{s \in S_m} \log P_\theta(\underbrace{x_i, \ldots x_j}_{s} \mid \mathbf{x}^{\text{mask}}) \qquad (7)$$

Here, θ denotes the model parameters, S_m is the set of selected masked statements, $\{x_i, \ldots, x_j\}$ represents the token sequence within the masked statement s, and $\mathbf{x}^{\text{mask}}$ is the masked input code sequence. Given that the TETransformer adopts the same architecture as CodeBERT, we can initially pre-train the TETransformer using MLM and then jointly pre-train both the TETransformer and SETransformer with MSP. The comparison between MSP and MLM is illustrated in Fig. 6. As shown, MSP employs an LSTM to predict the token sequence of a masked statement based on its feature vector, whereas MLM uses an FFNN to predict the masked tokens based on the token feature vectors generated by the TETransformer.

4.6 Design Rationale of Our Method

The design rationale behind our proposed CodeBERT-HLS is to capture both token-level and statement-level semantic information in source code, and make it suitable for multi-granular vulnerability detection. To achieve this, we introduce two key components: TETransformer and SETransformer. The TETransformer focuses on learning token-level semantics, treating each token as the smallest semantic unit, whereas the SETransformer targets capturing statement-level semantics, considering each statement as the fundamental unit. To effectively transfer semantic information from SETransformer to TETransformer, we develop three Token2Statement strategies. These strategies ensure a seamless connection between the two components, forming a cohesive and comprehensive architecture. A notable advantage of our architecture is that it produces an accurate representation of each statement, which provides the basis and opportunities to develop an automatic fine-grained vulnerability detection model. Based on the novel architecture, we design Algorithm 1 to enable the proposed architecture

to efficiently process long token sequences. In addition, we introduce a new pre-training task to help our architecture learn general program semantics, which gives it the ability to adapt quickly to vulnerability detection tasks and achieve better generalization in real-world scenarios.

5 Evaluation Setting

5.1 Research Questions

To evaluate the effectiveness of our method[1], we pose the following research questions (RQs). Given the significant contributions of CodeBERT-HLS and its pretraining task, we first assess their effectiveness in RQ1 and RQ2, respectively. In RQ3, we examine different components within CodeBERT-HLS. Next, in RQ4 and RQ5, we compare our approach's performance against state-of-the-art DL, transformer-based, and LLM-based methods in coarse-grained vulnerability detection. Finally, we demonstrate StagedVulBERT's significant performance enhancement in fine-grained vulnerability detection. The specific RQs are as follows:

RQ1: To what extent does the proposed CodeBERT-HLS demonstrate statistically significant superiority over CodeBERT in vulnerability detection?

RQ2: To what extent is the proposed pre-training task effective in improving model performance?

RQ3: To what extent does each component of the proposed CodeBERT-HLS contribute to its overall effectiveness?

RQ4: To what extent does the proposed StagedVulBERT outperform the state-of-the-art DL- and transformer-based methods in coarse-grained vulnerability detection?

RQ5: To what extent does the proposed StagedVulBERT outperform the current state-of-the-art LLM-based methods in coarse-grained vulnerability detection?

RQ6: To what extent does the proposed StagedVulBERT effectively enhance the performance of fine-grained vulnerability detection?

5.2 Dataset

The dataset big_vul we use in this paper is provided by Fan *et al.* [36], which is collected from 348 open-source projects. This dataset is notable for its data instances from various real-world application domains, including web browsers like Chrome, operating systems such as Linux, mobile platforms like Android, and software applications such as FFmpeg.Concretely, big_vul contains 188,636 samples, of which 10,900 are vulnerable functions and 177,736 are non-vulnerable functions. The vulnerable functions in the dataset are written in C/C++ and contain a total of 91 types of vulnerabilities, where CWE-119, CWE-20, CWE-399, CWE-264, CWE-416, CWE-200, CWE-125, CWE-189, CWE-362 and CWE-476 account for more than 60% of the entire data. The vulnerability categorization is based on the Common Weakness Enumeration (CWE) system. The distribution of vulnerabilities across different CWE types is shown in Fig. 7. To prepare the data for experiments, we follow the approach used in previous work [10] by randomly splitting the data into 80% for training, 10% for testing, and 10% for evaluation. Table 1 shows the statistics of the programs in the dataset, and Table 2 details the number of vulnerable and non-vulnerable programs in training, evaluation, and testing datasets.

1. All resources, including code, data, and models, are publicly available at https://github.com/YuanJiangGit/StagedVulBERT.git

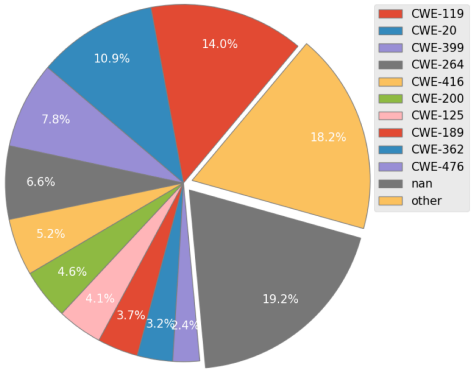


Fig. 7. The pie chart presents the distribution of vulnerability types of the programs in the dataset used in this paper. Each sector in the pie chart represents a different CWE whose size shows the proportion of programs with the specific type of vulnerabilities in the big_vul dataset.

TABLE 1
The statistics of the dataset used in this paper, where # iNum represents the number of programs in the dataset, # MeaL, # MinL, # MaxL, # MedL and # StdL denote the mean, minimum, median, maximum and standard deviation of the number of lines of code in the programs, respectively.

CWE ID	# iNum	# MeaL	# MinL	# MaxL	# MedL	# StdL
CWE-119	26,497	461.1	5	115,726	178.0	1,275.3
CWE-20	20,501	378.3	5	137,223	144.0	1,397.4
CWE-399	14,806	309.2	5	50,427	132.0	1,124.3
CWE-264	12,487	280.7	6	16,187	146.0	474.2
CWE-416	9,780	317.0	6	58,993	151.0	872.7
CWE-200	8,636	444.3	7	53,259	196.0	1,107.0
CWE-125	7,717	630.6	5	44,158	236.0	1,552.8
CWE-189	6,964	392.2	8	27,199	179.0	950.9
CWE-362	6,073	344.8	5	30,443	170.0	735.7
CWE-476	4,572	498.0	9	45,884	226.5	1,347.9
ALL	188,636	382.2	5	140,064	156.0	1202.5

TABLE 2
The number of vulnerable and non-vulnerable programs in training, evaluating, and testing datasets

Dataset	# Total	# Vul	# Non-Vul	# Vul(%)
Training Set	150,908	8,736	142,172	5.79
Evaluation Set	18,864	1,109	17,755	5.88
Testing Set	18,864	1,055	17,809	5.59
Total	188,636	10,900	177,736	5.78

5.3 Ground-truth Labels

The big_vul dataset from Fan *et al.* [36] is among the largest vulnerability datasets available, with each code function example labeled for both function-level and line-level ground truths. In this dataset, a code function within a project is labeled as vulnerable if it is modified in a vulnerability-fixing commit, which is recorded in a CVE entry. As in previous work [10], these modified lines are considered as line-level

labels for fine-grained vulnerability detection. Conversely, functions not modified in such commits are labeled as '0', indicating that they contain no vulnerabilities. The authors of big_vul have invested considerable manual effort in the data collection process to ensure the high quality of this dataset [37].

5.4 Baseline Methods

5.4.1 Methods for Coarse-grained Vulnerability Detection

To demonstrate the superiority of the proposed method, we consider the following types of approaches as baselines:

(1) **Token-based methods** first transform code snippets or code slices into sequences of tokens which are then fed into CNN- or RNN-based models for vulnerability detection. Frequently cited comparative methods in this category include the one by Russell *et al.* [3], VulDeePecker [8] and SySeVR [9].

(2) **GNN-based methods**, including notable approaches such as Devign [4], Reveal [5], IVDetect [6], and AMPLE [7], construct various code graphs and then utilize graph neural networks (GNNs) to learn vulnerability patterns from these graphs. Empirical studies [5], [7] have demonstrated that GNN-based methods outperform Token-based methods in vulnerability detection.

(3) **Transformer-based methods** employ pre-trained models to capture deep semantic features of code, making them particularly suitable for understanding complex vulnerabilities. Recent examples of such methods, LineVul [10], SVulD [12] and VulBERTa [11], have demonstrated promising results in identifying vulnerabilities within real-world datasets.

(4) **LLM-based methods** are trained with billions of parameters and training samples, which can also be explored in detecting vulnerabilities [16], [17]. In comparison, the recently proposed models such as DeepSeek-Coder [23], CodeLlama [24], StarCoder [25], WizardCoder [26], Mistral [27], Phi-2 [28], and ChatGPT [29] serve as baselines.

5.4.2 Methods for Fine-grained Vulnerability Detection

In this paper, we consider the following two types of fine-grained vulnerability detection methods as our baselines:

(1) **GNN-interpretation based methods** employ pre-trained GNN-based detection models to identify crucial features (e.g., nodes) for detection by assigning them importance scores, thereby achieving statement-level vulnerability prediction. In this paper, we select four GNN detectors (i.e., DeepWukong [38], Devign [4], IVDetect [6], and Reveal [5]) and six GNN interpreters (e.g., GNN-LRP [39], DeepLIFT [40], GradCam [41], GNNExplainer [42], PGExplainer [43], and SubGraphX [44]) collectively representing a wide spectrum of methods frequently used in this field. By pairing each detector with each interpreter, we consequently examine 24 distinct GNN-interpretation-based methods.

(2) **Attention-based methods**, such as LineVul [10], calculate the total score for a statement by summing the attention scores of all its tokens. This score serves as a criterion to evaluate the semantic importance of the statement line. These methods then rank statement lines based on their attention scores, under the assumption that lines with higher ranks play a decisive role in the model's decision-making process.

5.5 Parameter Configurations

In this paper, we pre-train CodeBERT-HLS using the widely-used CodeSearchNet dataset [45], which encompasses six different programming languages. Both the TETransformer and SETransformer in CodeBERT-HLS consist of six Transformer layers, each with a 768-dimensional hidden state and 12 self-attention heads. This configuration aims to facilitate a fair comparison with the 12-layer pre-trained CodeBERT model. Our pre-training is conducted on a computational platform equipped with five NVIDIA A100 GPUs. For pre-training StagedVulBERT, we adopt the following hyperparameter settings by conducting a random search in the hyperparameter space: a batch size of 400, a learning rate of 5e-5, a maximum input length of 512 tokens, and a maximum of 5 training epochs. Additionally, the AdamW optimizer is used to update the model parameters. Training 100K batches of data with MSP takes approximately 130 hours. Due to the high cost associated with searching for optimal hyperparameters on the entire CodeSearchNet dataset, we only validate these parameters on a small subset of the data to ensure they yield satisfactory performance compared to other parameter combinations.

After pre-training, we further fine-tune StagedVulBERT, integrating a classification network for vulnerability detection, using the big_vul dataset. This fine-tuning is performed on a server equipped with an A6000 (48G) GPU. The hyperparameters for fine-tuning are set as follows: a batch size of 32, a range of maximum input lengths among {512, 1024, 2048} tokens, and a learning rate of 2e-5. Similar to the pre-training phase, the AdamW optimizer is employed to update the model parameters. Completing a full fine-tuning cycle involves 10 epochs and takes approximately 10 hours. Note that we adopt a smaller batch size compared to pre-training, making it easier for readers to reproduce the fine-tuned model presented in this paper.

5.6 Evaluation Metrics

To evaluate the performance of our proposed method, we use the following five metrics, which have been widely accepted by previous work [9], [46].

- Precision (precision) is the fraction of predicted vulnerabilities that are correctly predicted [47], [48]; $precision = \frac{TP}{TP+FP}$.
- Recall (recall) is the fraction of true positive vulnerabilities in the actual vulnerabilities [47], [48]; $recall = \frac{TP}{TP+FN}$.
- F1 score is used to balance precision and recall. It is an aggregate way to describe a model [48]; $F1\ score = 2 \times \frac{precision \times (1-FNR).}{precision+(1-FNR)}$
- Accuracy (accuracy) is the percentage of correct predictions: $accuracy = \frac{TP+TN}{TP+FP+TN+FN}$.
- Top-k% Accuracy (Top-k% Acc) is the locating accuracy of a vulnerability detector e. It measures whether the top k% of statements predicted as vulnerable by the detector are correct by checking if they overlap with the fine-grained ground truth. The formula is as follows, where r_i denotes the fine-grained

label of sample $x_i \in D$ and p_i is the prediction of the top k% most vulnerable statements.

$$Acc(e, D, k) = \frac{1}{|D|} \sum_{x_i \in D} Acc\,(e, x_i, k)$$
$$\text{Acc}\,(e, x_i, k) = \begin{cases} 1, r_i \cap p_i \neq \emptyset \\ 0, r_i \cap p_i = \emptyset \end{cases} \quad (8)$$

6 Evaluation Results

6.1 Experiments for Answering RQ1

CodeBERT, as employed in the recently proposed LineVul method by Fu et al. [10] and the VulBERTa method by Hanif et al. [11], has been successfully applied to the task of vulnerability detection, achieving satisfactory performance. Given that both our proposed model CodeBERT-HLS and CodeBERT are based on a transformer-encoder architecture and adhere to a similar learning protocol of extensive pre-training on large datasets via self-supervised learning followed by fine-tuning for vulnerability detection, this section aims to determine whether our model exhibits a statistically significant advantage over CodeBERT.

To address these questions, we randomly divide the big_vul data into ten disjoint, equal-sized sets, named $fold_1$, $fold_2$, ..., $fold_{10}$. We then conduct cross-validation, training our method on $fold_i$, evaluating it on $fold_{i+1}$, and testing it on $fold_{i+2}$. Since our method accommodates input sequences of varying token lengths, we also compare different variants of our approach, specifically StagedVulBERT (512), StagedVulBERT (1024), and StagedVulBERT (2048). These variants employ the same network architecture, CodeBert-HLS with the 'Average Token2Statement', but differ only in the length of input samples processed, with specific token counts of 512, 1024, and 2048, respectively. For each method, we obtain eight values of accuracy, recall, precision and F1 score corresponding to the eight testing sets.

Taking the overall effectiveness, as measured by the F1 score, as an example: without pre-training, our methods StagedVulBERT (512), StagedVulBERT (1024), and StagedVulBERT (2048) show an improvement over CodeBERT by 74.40%, 88.87%, and 96.97% respectively in terms of average performance, as illustrated in Fig. 8. This suggests that our proposed model architecture is more suited for the vulnerability detection task compared to the CodeBERT model.

To determine whether our method demonstrates a statistically significant advantage, we perform a two-tailed Wilcoxon Signed-Rank Test [49] to analyze the differences between pairs of results from each sub-figure in Figs. 8. Recall that a significance level of 0.05 or less (indicating 95% confidence) denotes statistical significance. We observe that all the T statistic for Wilcoxon's test (T_{wilcox}) are smaller than the critical value as provided by [50], suggesting that our model architecture has a statistically significant advantage.

Conclusion: Without pre-training, StagedVulBERT with CodeBERT-HLS outperforms LineVul with CodeBERT, as CodeBERT-HLS has a more suitable network architecture for vulnerability detection tasks.

6.2 Experiments for Answering RQ2

RQ1 has demonstrated the effectiveness of the proposed CodeBERT-HLS; this section investigates the role of the pre-training task introduced in this paper in enhancing CodeBERT-HLS's capability for vulnerability detection. To achieve this, we use the steps in Section 4.5 and the hyper-parameters detailed in Section 5.5 to pre-train CodeBERT-HLS. We then compare the performance differences of StagedVulBERT with the pre-trained or non-pre-trained CodeBERT-HLS in coarse-grained vulnerability detection. Finally, we perform cross-validation with ten equally sized subsets from $fold_1$ to $fold_{10}$, which is inconsistent with the experimental settings in RQ1. We also investigate the effect of varying input sequence lengths (512, 1024, 2048) on the performance of both pre-trained and non-pre-trained CodeBERT-HLS models. The experimental results are provided in Fig. 9.

As shown in Fig. 9, when initialized with pre-trained weights, StagedVulBERT (512), StagedVulBERT (1024), and StagedVulBERT (2048) achieve average F1 scores of 78.94%, 82.63%, and 85.50% respectively across eight testing sets. These scores represent improvements of 28.19%, 23.91%, and 22.94% over the variants without pre-training. This indicates that the proposed pre-training task MSP effectively assists the model in learning more generalized feature representations of programs, thereby enhancing performance in downstream vulnerability detection.

Furthermore, to determine whether the performance improvements are statistically significant, we conduct the standard Wilcoxon Signed-Rank Test at a 0.05 significance level for pairs of results from each sub-figure in Fig. 9. We find that all T-statistics for Wilcoxon's test (T_{Wilcox}) are below the critical value, as per [50], indicating that our pre-training task is effective and the results are statistically significant. Given that StagedVulBERT (2048) with pre-training shows the best performance, it has been selected as our primary method for subsequent experiments.

Conclusion: With pre-training, StagedVulBERT with CodeBERT-HLS exhibits significantly enhanced effectiveness compared to its version without pre-training. This emphasizes the importance of pre-optimization through the proposed MSP task to achieve the best performance.

6.3 Experiments for Answering RQ3

The effectiveness of the entire architecture of CodeBERT-HLS has been demonstrated previously in RQ1. In this section, we will investigate the effectiveness of each component (i.e., TETransformer, Token2Statement Embedding, and SETransformer) separately. The TETransformer, sharing the same architecture with CodeBERT, can be directly utilized for vulnerability detection without modification. However, the SETransformer, which takes the initial representation of each statement as input, requires adaptation for vulnerability detection. To achieve this, the average embedding of all tokens within a statement is employed as input. In addition, relying solely on Token2Statement is insufficient for direct vulnerability detection, as it is situated in an intermediate layer and cannot directly produce predictive output. Thus, we analyze the performance of different design choices for Token2Statement embedding with the same SETransformer

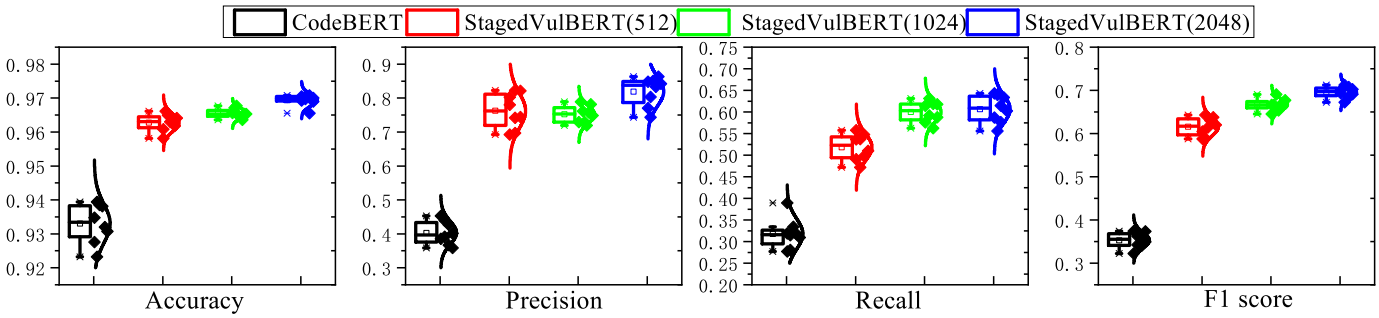


Fig. 8. Performance comparison of StagedVulBERT (with input lengths of 512, 1024, and 2048) and CodeBERT in coarse-grained vulnerability detection without pre-training using boxplots. Each boxplot represents the performance distribution of each method on split datasets. StagedVulBERT variants differ in their maximum input length: 512, 1024, and 2048. For example, StagedVulBERT (512) has a maximum input length of 512.

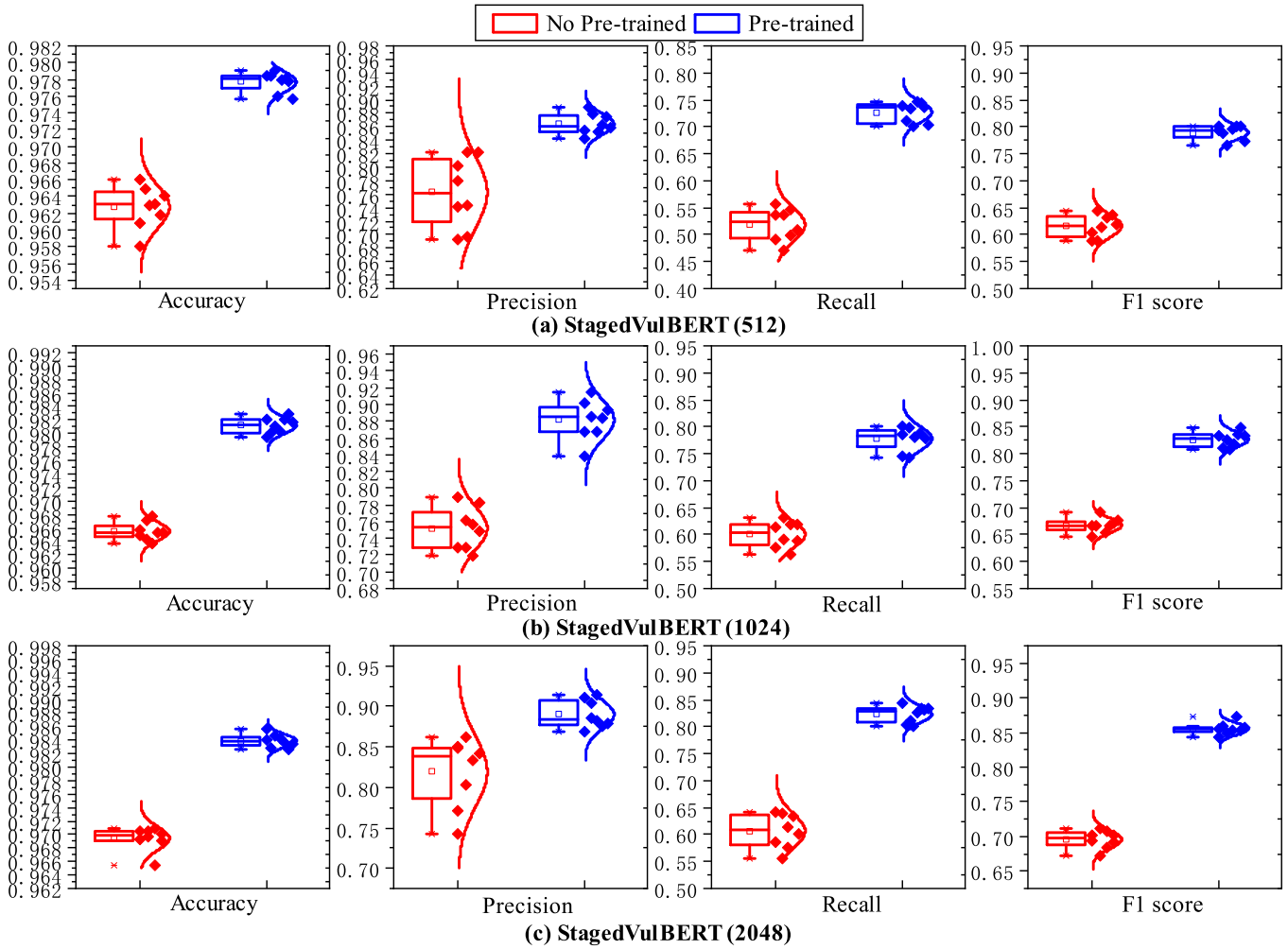


Fig. 9. Performance comparison of Pre-trained StagedVulBERT versus No Pre-trained StagedVulBERT in coarse-grained vulnerability detection using boxplots. The first row of subfigures presents experiments conducted with a 512 input length limit, while the second and third rows display the results for the 1024 and 2048 input lengths, respectively. Each boxplot represents the performance distribution of each method on the split datasets.

and TETransformer. The detailed experimental results of our method's variants with pre-training are presented in Table 3.

TABLE 3
Performance comparison of our method's variants: TETransformer (TE), SETransformer (SE), and their combination with three Token2Statement strategies—Average (Ave), Weighted Average (Wei), and Attention Average (Alt) Token2Statement.

Approach	Accuracy	Precision	Recall	F1 score
Only use TETransformer (TE) or SETransformer (SE)				
TE	97.84	88.03	71.09	78.66
SE	**97.92**	86.94	**73.84**	**79.86**
TE and SE with different Token2Statement				
TE+SE+Ave	**99.15**	**94.26**	90.33	**92.26**
TE+SE+Wei	99.12	93.88	90.14	91.97
TE+SE+Att	99.10	92.59	**91.18**	91.88

As can be seen from Table 3, when employing solely the TETransformer or SETransformer, a strong drop in detection performance is observed due to reduced program comprehension ability. Additionally, combining TETransformer and SETransformer with "Average Token2Statement" achieves a better performance than the combination with "Weighted Average Token2Statement". This indicates that determining the semantic importance of a token based on its absolute position during Token2Statement stage does not enhance the model's code representation capability. This is because positional embeddings in the TETransformer sufficiently capture token locations, which are vital for semantic understanding in subsequent layers. Adding positional weighting in the Token2Statement embedding does not consistently boost performance; thus, the "Weighted Average Token2Statement" method, duplicating absolute position capture, offers no benefits.

Among all the variants, the one employing the "Attention Average Token2Statement" strategy shows the poorest performance. One possible explanation is that the attention scores might not accurately reflect the true significance of tokens for the given task. This finding is consistent with similar research, such as LineVul, which also tried to use token attention scores for identifying potentially vulnerable statements but faced limitations. As demonstrated in Section 2.1, relying solely on token attention scores fails to precisely pinpoint vulnerabilities at a fine-grained level. Hence, it appears that at the present scale of training data, the attention scores output by the PCL model for tokens may not accurately represent the semantic importance of these tokens, and could even lack a direct correlation with their semantics. This observation helps to understand why the "Attention Average Token2Statement" variant falls short in performance when compared to others.

Conclusion: Our investigation into StagedVulBERT's components demonstrates their effectiveness in detecting vulnerabilities. Combining these components achieves optimal performance, particularly when employing the "Average Token2Statement" in the intermediate layer.

6.4 Experiments for Answering RQ4

In addition to demonstrating the effectiveness of the model architecture and pre-training task discussed in RQ1 and RQ2, respectively, we also aim to assess whether our method outperforms current state-of-the-art approaches in coarse-grained vulnerability detection. To answer this question, we compare our approach with ten baselines including three RNN-based methods, four GNN-based methods and three transformer-based methods. Our StagedVulBERT approach, with a 2048 input token limit, is pre-trained on CodeSearchNet and fine-tuned on the training set of big_vul. Table 4 shows the experimental results on the big_vul dataset.

TABLE 4
Comparisons between our method and others in coarse-grained vulnerability detection when evaluated on the big_vul dataset

Type	Baseline	Accuracy	Precision	Recall	F1 score
RNN-based methods	VulDeePecker	87.19	38.44	12.75	19.15
	Russell *et al.*	86.85	14.86	26.97	19.17
	SySeVR	90.10	30.91	14.08	19.34
GNN-based methods	Devign	92.78	30.61	15.96	20.98
	Reveal	87.14	17.22	34.04	22.87
	IVDetect	-	23	72	35
	AMPLE	93.14	29.98	34.58	32.11
Transformer-based methods	VulBERTa	93.99	48.44	33.54	39.64
	SVulD	97.1	72.4	79.3	75.7
	LineVul	98.53	94.2	78.58	85.68
	StagedVulBERT	**99.15**	**94.26**	**90.33**	**92.26**

Table 4 shows that Transformer-based methods perform better than other DL-based models. This is consistent with the observations by previous work which confirms the powerful capability of Transformer-based methods for automatic vulnerability feature extraction and representation [10]. Among existing methods, LineVul exhibits the best performance with an F1 score of 85.68%, though this falls short of the 91.0% reported in the original paper. This discrepancy is due to the original implementation of LineVul removing all code indentation from the vulnerable programs in the big_vul dataset during its data preprocessing phase. This leads to a format difference between vulnerable and non-vulnerable code, thereby reducing the difficulty of identifying vulnerable code and resulting in a higher F1 score. To more accurately reflect real-world scenarios, we address this issue and re-conduct experiments on the original dataset using the publicly available implementation of LineVul without such preprocessing.

Compared to the state-of-the-art Transformer-based methods, StagedVulBERT demonstrates notable performance improvements. It achieves an 11.75% increase in recall and a substantial 6.58% improvement in F1 score over LineVul. While accuracy sees a slight increase of 0.62%, the overall performance gain is evident, particularly in the critical metrics of recall and F1 score, which are vital for a comprehensive evaluation of vulnerability detection models. The higher detection performance could be due to the fact that the proposed CodeBERT-HLS used in StagedVulBERT is more capable of learning semantic and syntactic of code, particularly for larger programs, compared to other methods (e.g., LineVul and SVulD) that are limited to processing input sequences of no more than 512 tokens.

To further investigate whether our method can effectively process a greater number of real-world programs compared to methods like LineVul, we conduct a statistical analysis examining the number of truncated samples under various input length constraints. As illustrated in Fig. 10

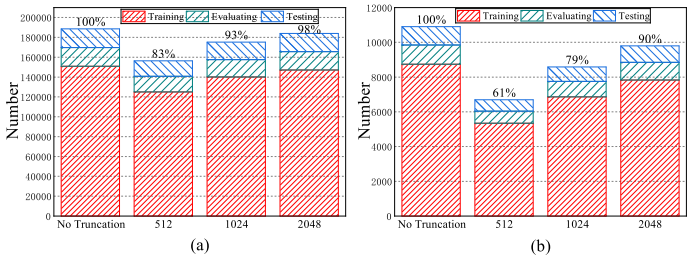


Fig. 10. Number of untruncated samples (Vertical axis) in the big_vul dataset at different truncation lengths (Horizontal axis). The left graph (a) shows statistics for the entire dataset, while the right graph (b) displays results specifically for the vulnerable data in the big_vul.

(left), with a 512-token limit, 17% of samples in the big_vul dataset are truncated, leading to potential information loss. However, increasing the token limit to 1024 and 2048 reduces truncation to 7% and 2%, respectively, thus preserving more of the original code information. Moreover, when focusing specifically on the vulnerable functions in the big_vul dataset (Fig. 10 (right)), only 61% of vulnerable programs are fully processed by the model when truncated at 512 tokens. This percentage increases to 79% and 90% when the input length is extended to 1024 and 2048 tokens, respectively. Consequently, with its ability to handle longer sequences, CodeBERT-HLS learns more comprehensive code features, particularly those from complex vulnerability code, thereby boosting its vulnerability detection capabilities.

Conclusion: Our method achieves a greater improvement than the state-of-the-art DL-based model, especially in the F1 score (from 85.68% to 92.26%). This could be due to the fact that our method can correctly learn code vector representations from the vulnerability dataset.

6.5 Experiments for Answering RQ5

LLMs have shown outstanding performance in various code-related tasks, such as code generation and code search. This section aims to explore whether LLMs also exhibit similar excellence in the vulnerability detection task. We also wonder whether our proposed method provides superior vulnerability prediction compared to those LLM-based methods. This exploration would help determine the necessity for further research into specific transformer-based methods and technologies.

To evaluate the performance differences between StagedVulBERT and LLMs in coarse-grained vulnerability detection, we conduct experiments on the big_vul dataset. The experiments involve comparisons with six recently proposed open-source LLMs: DeepSeek-Coder 6.7B, CodeLlama 7B, StarCoder 7B, WizardCoder 7B, Mistral 7B, and Phi-2 2.7B, as well as two close-source LLMs from OpenAI: Chatgpt-3.5 and Chatgpt-4. To make these open-source LLMs align with the vulnerability detection task, we employ the "AutoModelForSequenceClassification" from the Transformers library[2] to implement discriminative vulnerability detection. This model class seamlessly appends a classification layer on top of each pre-trained LLM, adapting it for sequence classification tasks. We fine-tune this model using the "big_vul" dataset, enabling LLMs to accurately classify code inputs as "vulnerable" or "not vulnerable". The above experimental settings are consistent with recent state-of-the-art empirical research [16]. For ChatGPT models, we use a few-shot setting to construct prompts designed for vulnerability detection, as detailed in related work [17]. The results of these experiments are presented in Table 5. From these, we can draw the following conclusions:

2. https://pypi.org/project/transformers/, a Python library

TABLE 5
Performance comparison of LLM-based methods and StagedVulBERT in coarse-grained vulnerability detection when evaluated on the big_vul dataset

Type	Baseline	Accuracy	Precision	Recall	F1 score
LLM-based (Business)	Chatgpt-3.5	0.907	0.115	0.912	0.103
	Chatgpt-4	0.911	0.251	0.274	0.292
LLM-based (Open-source)	DeepSeek-Coder	0.964	0.633	0.886	0.739
	CodeLlama	0.958	0.598	0.850	0.702
	StarCoder	0.955	0.579	0.784	0.666
	WizardCoder	0.955	0.574	0.882	0.695
	Mistral	0.919	0.405	0.873	0.553
	Phi-2	0.955	0.569	0.896	0.696
Our methods	StagedVulBERT	**0.992**	**0.943**	**0.903**	**0.923**

Our proposed StagedVulBERT significantly outperforms LLM-based methods in terms of all evaluation metrics, with the latter showing relatively weaker performance. For instance, StagedVulBERT achieves an F1 score of 92.3%, while the better-performing baseline (i.e., DeepSeek-Coder) only reaches 73.9%. In addition, LLMs fine-tuned for vulnerability detection perform better than those LLMs (e.g., Chatgpt-4) in the few-shot setting, as observed in [16]. These results suggest that despite LLM's excellence in natural language processing, it still falls behind models specifically designed for vulnerability detection, such as StagedVulBERT. This also demonstrates the significance of continuing research into PCL models for the vulnerability detection task.

Conclusion: LLM can be used in detecting software vulnerabilities, but fine-tuned open-source LLMs and few-shot commercial LLMs perform weaker than our method.

6.6 Experiments for Answering RQ6

To demonstrate the superiority of our proposed method in fine-grained vulnerability detection, we compare it with existing fine-grained vulnerability detection baselines, as outlined in Section 5.4.2, using the big_vul dataset annotated with fine-grained labels. Our evaluation metric involves calculating the accuracy of the model's predictions for the top k% of code lines, with k ranging from 2 to 20. The experimental results are depicted in Fig. 11. From these results, we can draw the following conclusions:

StagedVulBERT, proposed in this paper, significantly outperforms other methods in fine-grained vulnerability detection. When compared with GNN-interpretation-based methods and the attention-based method, StagedVulBERT consistently shows superior accuracy across various settings of k. For instance, at k=5, the best-performing baseline method, LineVul, achieves an accuracy of 37.5%, whereas StagedVulBERT reaches an impressive 65.69% under the same conditions, marking a performance increase of 75.17%. This substantial improvement is largely due to the fact

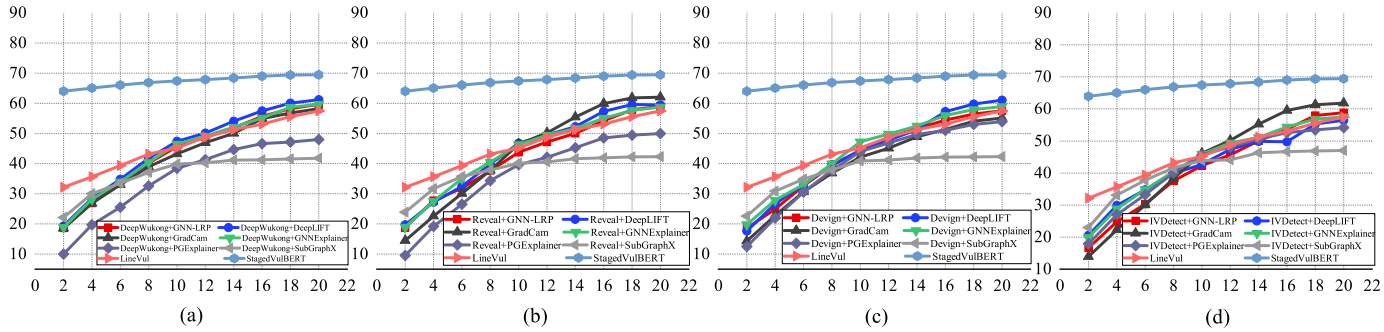


Fig. 11. Comparisons between our method and others in fine-grained vulnerability detection when evaluated on the big_vul dataset. The horizontal axis represents the *k* values ranging from 2 to 20, while the vertical axis shows the accuracy of the Top-*k*% predicted statements. Due to a total of 24 GNN-interpretation based methods in the baselines, which is too numerous to display in a single figure, we have evenly distributed them across four sub-figures: (a), (b), (c), and (d).

that previous studies focus on training models for coarse-grained detection using only coarse-grained labels via calculating maximum likelihood, and then relying on interpreters or attention scores to assess the contribution of each code line to the model's output. In contrast, StagedVulBERT, after initially training a coarse-grained vulnerability detection model, employs supervised learning to further train a model specifically designed for vulnerable statement classification. This allows it to directly determine the relevance of a statement to vulnerabilities based on its feature representation. By effectively leveraging both coarse- and fine-grained labels during training, StagedVulBERT significantly enhances its ability to precisely locate specific lines of vulnerable code.

Another factor contributing to StagedVulBERT's high performance is the proposed CodeBERT-HLS's ability to generate accurate statement feature representations. By employing the self-attention mechanism of the SETransformer, it captures the interdependencies between statements. This means that the generated statement features contain not only the semantics of the statement itself but also an integrated context of related information. Moreover, as the SETransformer can process up to 512 lines of statements, it captures a longer range of dependencies than other token-based models. Therefore, even though our statement classifier utilizes a simple feedforward neural network, it still achieves excellent experimental results. This approach, which considers both the local semantic information of statements and the overall structural dependencies in the code, provides stronger support for accurately locating and understanding vulnerabilities.

Conclusion: Compared to the baselines, our proposed approach demonstrates superior effectiveness in locating vulnerable lines of code, which achieves a higher locating precision compared to the state-of-the-art fine-grained vulnerability detection methods.

7 Discussion

7.1 How Accurate is Our StagedVulBERT in Predicting the Most Frequent and Dangerous CWEs?

To validate the practicality of StagedVulBERT in real-world scenarios, we conduct experiments on 14 types of CWE code samples, which are the most frequent and dangerous in the big_vul dataset. Among these, CWE-119, CWE-20, CWE-399, CWE-264, CWE-416, CWE-200, CWE-125, CWE-189, CWE-362 and CWE-476 are the most commonly occurring CWE types in the big_vul dataset, while CWE-787, CWE-125, CWE-20, CWE-416, CWE-22, CWE-190, CWE-476, CWE-119 and CWE-77 are recognized among the Top-25 most dangerous CWEs [51]. For each of these 14 CWE types, we evaluate StagedVulBERT's performance in both coarse- and fine-grained vulnerability detection.

The results, presented in Fig. 12, show that in coarse-grained detection, StagedVulBERT achieves average accuracy, precision, and F1 score of 99.19%, 94.95%, and 92.81%, respectively. Notably, StagedVulBERT shows weaker performance in detecting CWE-22 (path traversal) vulnerabilities, with an F1 score of only 85.71%, but excelled at detecting CWE-77 (improper neutralization) vulnerabilities, achieving an F1 score of 100%. In fine-grained detection, StagedVulBERT also demonstrates superior performance, with Top-5% and Top-10% accuracy rates of 64.33% and 65.42%, respectively. This further confirms its significant advantages in precisely identifying and locating specific vulnerabilities in code, particularly in scenarios involving a range of different vulnerability types.

Furthermore, we present a comparison of the F1 score performance between the proposed StagedVulBERT and the state-of-the-art method, LineVul with CodeBERT, on the most frequent and dangerous vulnerability samples. As illustrated in Fig. 13, StagedVulBERT achieves performance improvements in all these CWEs. For instance, the improvements in CWE-22 (path traversal) and CWE-77 (improper neutralization) vulnerabilities are 28.56% and 50.0%, respectively. Overall, StagedVulBERT's average F1 score on these 14 CWEs of samples shows a 7.33% increase compared to CodeBERT.

7.2 Investigate Whether the Proposed StagedVulBERT Can Generalize to Other Datasets

The big_vul dataset, widely used in recent studies due to its large size and real-world project origins, accurately reflects sample imbalance in actual scenarios, thus providing a more authentic experimental environment for vulnerability detection. Even with this, we would like to explore whether our method can generalize well to other datasets. To achieve this, we select a new dataset publicly published by [34],

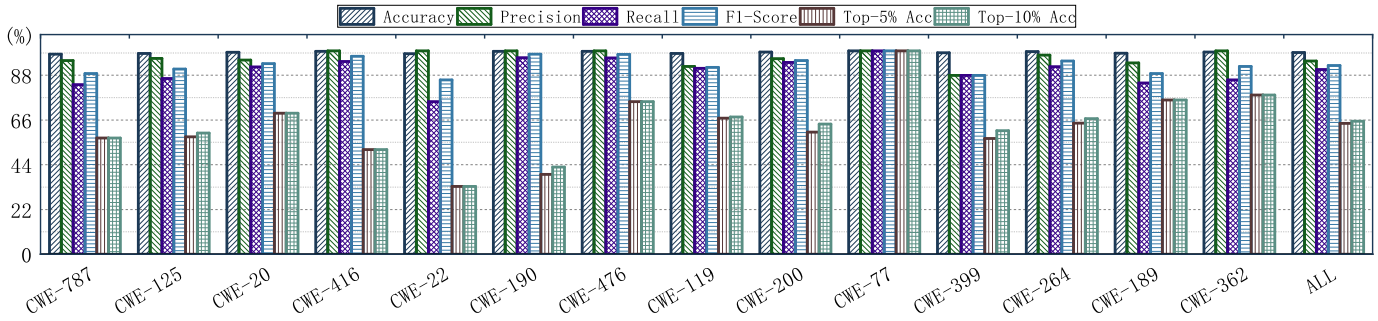


Fig. 12. The performance of our proposed method StagedVulBERT to detect vulnerabilities of real-world programs with the top-14 CWE vulnerability types.

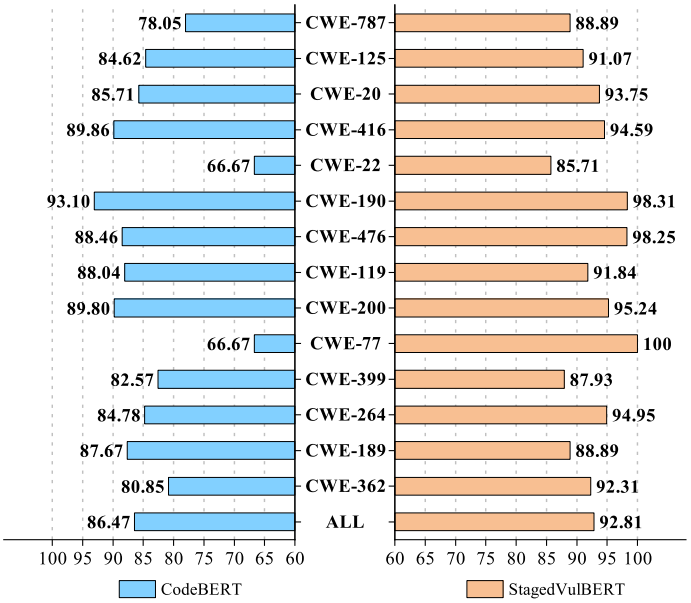


Fig. 13. The accuracy of our method and the state-of-the-art method for the Top-14 common and high-risk CWEs

which consists of 14,511 programs, each labeled with detailed vulnerability information down to the line level. The experimental setting is the same as detailed in Section 5.5 and the experimental results are shown in Table 6.

TABLE 6
Comparisons between our method and LineVul when evaluated on the dataset used in [34]

Model	Coarse-grained				Fine-grained	
	Accuracy	Precision	Recall	F1 score	Top-5% Acc	Top-10% Acc
LineVul	92.29	**84.78**	84.86	84.82	21.00	25.00
StagedVulBERT	**92.73**	83.51	**88.93**	**86.14**	**94.57**	**95.54**

As can be seen in Table 6, on the dataset published by [34], our proposed StagedVulBERT outperforms LineVul in both coarse- and fine-grained vulnerability detection tasks, with the latter particularly underperforming in fine-grained detection. For instance, in fine-grained detection, StagedVulBERT's Top-10% Accuracy is 95.54%, significantly higher than the 25.00% of the LineVul, indicating a substantial gap. These results suggest that the statement vectors generated by our proposed CodeBERT-HLS effectively capture the semantics of statements and are better suited as input for fine-grained vulnerability detection models compared to the token vectors produced by the baseline method.

7.3 Investigate the Cases where StagedVulBERT Model Failed to Detect Vulnerabilities

As seen in the results of our previous experiment, the proposed StagedVulBERT achieves an overall F1 score of 92.26% for coarse-grained vulnerability detection on the big_vul dataset, which is the best performance achieved so far. That means our method can identify most vulnerabilities, including those undetectable by LineVul as shown in Fig. 1. However, there are still some vulnerabilities our method fails to detect. For example, the "get_control" function shown in Fig. 14 retrieves a control structure for PNG file handling, but it improperly casts pointers using png_voidcast, leading to potential memory corruption or privilege gains. Thus, this function is a vulnerable example. However, our method fails to detect the vulnerability because it spans two functions (i.e., "get_control" and "png_voidcast"), which makes it difficult or even impossible to detect by analyzing each function individually. This limitation also exists in other transformer-based methods, which we will leave as future work by considering the dependencies among functions.

```
1 get_control(png_const_structrp png_ptr)
2 {
3     /* This just returns the (file*).  The chunk and idat control structures
4      * don't always exist.
5      */
6     struct control *control = png_voidcast(struct control*,
7        png_get_error_ptr(png_ptr));
8     return &control->file;
9 }
```

Fig. 14. A real-world vulnerability (i.e., CVE-2016-3751) in Android 4.x, allowing attackers to gain privileges via a crafted application, was not detected by our proposed method

8 Limitations and Future Work

This paper proposes a novel detection framework StagedVulBERT and demonstrates its effectiveness when compared to DL-, transformer-, and LLM-based methods. However, it still has some limitations. First, while the big_vul dataset is preferred by many baseline methods due to its real-world project origins and reflective sample imbalance, its

labels, derived from bug-fix commits, might introduce noise that can impact model performance. Although this issue is recognized, as discussed in [52], no ideal solution exists yet. Future work will focus on enhancing data quality research to minimize the influence of data noise on model training, as well as conducting more experiments on other datasets to evaluate the generalization ability of the proposed method. Second, although our method performs well in detecting vulnerabilities related to C/C++, its effectiveness for other languages, such as Java, is yet to be determined. In future work, we plan to extend our approach to other programming languages. Since our approach does not rely on complex program parsing or compilation processes, it can theoretically be easily adapted to other programming languages. Third, while our method can process longer code sequences through the proposed Algorithm 1, we currently truncate code samples when they exceed the token limit. This solution may not be optimal. Future work could explore using specific filtering heuristics to preserve more useful information and improve performance. In addition, future research could benefit from incorporating contextual information such as AST, CFG, and PDG for enhanced vulnerability detection. Although it has been emphasized in many studies, most perform experiments on only a small fraction of functions in the big_vul dataset, as numerous functions cannot be accurately parsed by existing tools such as Joern [12]. Therefore, combining this information with transformer architectures still presents challenges in real-world applications but also holds promise for more accurate, fine-grained detection.

9 Related work

With large amounts of code publicly available and the development of machine learning and DL technology, data-driven vulnerability discovery sheds new light on intelligent, automatic, and effective vulnerability detection. DL-based detection methods (e.g., VulDeePecker [8] and SySeVR [9]) directly learn vulnerability patterns from the original source code, which can reduce the time and labor cost significantly due to no need for human-defined features. In addition, since many vulnerabilities can only be effectively detected by simultaneously considering the structure, control flow and dependencies within code, researchers have proposed several graph neural network-based models (e.g., Devign [4], Vu1SPG [53], DeepWukong [38], GINN [54], Reveal [5] and AMPLE [7]) to learn a rich set of code semantics from these representations. They claim that their preliminary experiments show promising results, but there remains potential for improvement in vulnerability detection of big functions with excessively long tokens [5]. In recent years, some transformer-based methods (e.g., LineVul [10], VulBERTa [11] and SVulD [12]) have been proposed for vulnerability detection and have achieved great performance. This is supported by a review paper [13] which compared 11 cutting-edge deep learning frameworks, highlighting LineVul's significant advantages on well-known datasets such as Devign and Big-Vul. LLMs have also been explored for this task, but their performances are weaker than those of specific transformer-based methods. Empirical studies [16] have confirmed the aforementioned findings.

In addition, the granularity of the aforementioned DL-based vulnerability detection methods is relatively coarse, primarily enabling detection at the function or code snippet levels. To achieve fine-grained vulnerability detection, some studies have proposed the following approaches: (1) interpreting prediction results using the attention mechanism employed in the coarse-grained detection model architecture [10]; (2) reusing statement representations extracted via GAT to provide statement-level detection [37]; (3) using gradients or hidden feature mapping values as approximations of input importance to identify tokens that may lead to vulnerabilities [55], [56]. However, these methods often struggle with generalization across different models. Therefore, some research has introduced the following interpretability methods, each specifically crafted to be compatible with a range of coarse-grained detection approaches. For instances, Studies [6], [42], [43], [57], [58], [59] explore how detection model outputs change when inputs are perturbed, thereby identifying inputs that significantly contribute to the predictions. Other research, such as [39], [60], involves decomposing a model's predictions into several components to evaluate the importance of input features. Additionally, there are approaches that train surrogate models, which mimic the behavior of the more complex original models, thus providing insights into the prediction mechanisms of these original detection models [61], [62], [63].

10 Conclusion

In this paper, we introduce a PCL-based vulnerability detection framework targeted at both coarse- and fine-grained level detection. We first leverage the proposed CodeBERT-HLS to automatically learn representations for code samples at different levels of semantic abstraction. Then, based on the learned representations, we develop a coarse-grained and a fine-grained vulnerability detection model based on DL. Benefiting from our novel PCL model and the pre-training task, our framework demonstrates significantly promising performance on a large-scale real-world dataset, as evidenced by our empirical experiments. Notably, in fine-grained vulnerability detection, our method achieves a 75.17% improvement in Top-5% accuracy compared to the state-of-the-art method.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant Nos.62302125 and 62272132) and the Heilongjiang Postdoctoral Fund (No.LBH-Z23019).

References

[1] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," *Proceedings - IEEE Symposium on Security and Privacy*, pp. 590–604, 2014.

[2] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953.

[3] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," pp. 757–762, 2018.

[4] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems*, 2019, pp. 10 197–10 207.

[5] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Transactions on Software Engineering*, 2021.

[6] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 292–303.

[7] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J. M. Zhang, and Q. Liao, "Vulnerability detection with graph simplification and enhanced graph representation learning," *arXiv preprint arXiv:2302.04675*, 2023.

[8] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, "Vulpecker: an automated vulnerability detection system based on code similarity analysis," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016, pp. 201–213.

[9] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, 2021.

[10] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.

[11] H. Hanif and S. Maffeis, "Vulberta: Simplified source code pre-training for vulnerability detection," in *2022 International joint conference on neural networks (IJCNN)*. IEEE, 2022, pp. 1–8.

[12] C. Ni, X. Yin, K. Yang, D. Zhao, Z. Xing, and X. Xia, "Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1611–1622.

[13] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2237–2248.

[14] Q. Cui, S. Huang, J. Li, X. Geng, Z. Zheng, G. Huang, and J. Chen, "Directqe: Direct pretraining for machine translation quality estimation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 14, 2021, pp. 12 719–12 727.

[15] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," vol. EMNLP 2020, pp. 1536–1547, 2020.

[16] X. Yin and C. Ni, "Multitask-based evaluation of open-source llm on software vulnerability," *arXiv preprint arXiv:2404.02056*, 2024.

[17] M. Fu, C. Tantithamthavorn, V. Nguyen, and T. Le, "Chatgpt for vulnerability detection, classification, and repair: How far are we?" *arXiv preprint arXiv:2310.09810*, 2023.

[18] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 837–847.

[19] H. K. Dam, T. Tran, T. T. M. Pham, S. W. Ng, J. Grundy, and A. Ghose, "Automatic feature learning for predicting vulnerable software components," *IEEE Transactions on Software Engineering*, 2018.

[20] H. K. Dam, T. Tran, and T. Pham, "A deep language model for software code," *arXiv preprint arXiv:1608.02715*, 2016.

[21] V. J. Hellendoorn and P. Devanbu, "Are deep neural networks the best choice for modeling source code?" in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 763–773.

[22] S.-R. Lee, M.-J. Heo, C.-G. Lee, M. Kim, and G. Jeong, "Applying deep learning based automatic bug triager to industrial projects," in *Proceedings of the 2017 11th Joint Meeting on foundations of software engineering*, 2017, pp. 926–931.

[23] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[24] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[25] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[26] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," *arXiv preprint arXiv:2306.08568*, 2023.

[27] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[28] M. Javaheripi, S. Bubeck, M. Abdin, J. Aneja, S. Bubeck, C. C. T. Mendes, W. Chen, A. Del Giorno, R. Eldan, S. Gopi *et al.*, "Phi-2: The surprising power of small language models," *Microsoft Research Blog*, 2023.

[29] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[30] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[31] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[32] P. Jain, A. Jain, T. Zhang, P. Abbeel, J. E. Gonzalez, and I. Stoica, "Contrastive code representation learning," *arXiv preprint arXiv:2007.04973*, 2020.

[33] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[34] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "Vuldeelocator: a deep learning-based fine-grained vulnerability detector," *IEEE Transactions on Dependable and Secure Computing*, 2021.

[35] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[36] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "Ac/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.

[37] D. Hin, A. Kan, H. Chen, and M. A. Babar, "Linevd: Statement-level vulnerability detection using graph neural networks," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 596–607.

[38] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.

[39] T. Schnake, O. Eberle, J. Lederer, S. Nakajima, K. T. Schütt, K.-R. Müller, and G. Montavon, "Higher-order explanations of graph neural networks via relevant walks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 11, pp. 7581–7596, 2021.

[40] A. Shrikumar, P. Greenside, and A. Kundaje, "Learning important features through propagating activation differences," in *International conference on machine learning*. PMLR, 2017, pp. 3145–3153.

[41] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 618–626.

[42] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "Gnnexplainer: Generating explanations for graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[43] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, and X. Zhang, "Parameterized explainer for graph neural network," *Advances in neural information processing systems*, vol. 33, pp. 19 620–19 631, 2020.

[44] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, "On explainability of graph neural networks via subgraph explorations," in *International conference on machine learning*. PMLR, 2021, pp. 12 241–12 252.

[45] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[46] Y. Hu, S. Wang, W. Li, J. Peng, Y. Wu, D. Zou, and H. Jin, "Interpreters for gnn-based vulnerability detection: Are we there yet?" in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1407–1419.

[47] J. Davis and M. Goadrich, "The relationship between precision-recall and roc curves," in *Proceedings of the 23rd international conference on Machine learning*, 2006, pp. 233–240.

[48] D. M. Powers, "Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation," 2011.

[49] F. Wilcoxon, "Individual comparisons by ranking methods," in *Breakthroughs in statistics*. Springer, 1992, pp. 196–202.

[50] N. Japkowicz and M. Shah, *Evaluating learning algorithms: a classification perspective*. Cambridge University Press, 2011.

[51] Homeland Security Systems Engineering and Development Institute operated by MITRE, "2023 CWE Top 25 Most Dangerous Software Weaknesses," https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html, 2023, accessed: 2024-02-17.

[52] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.

[53] W. Zheng, Y. Jiang, and X. Su, "Vu1spg: Vulnerability detection based on slice property graph representation learning," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 457–467.

[54] Y. Wang, K. Wang, F. Gao, and L. Wang, "Learning semantic program embeddings with graph interval neural network," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–27, 2020.

[55] F. Baldassarre and H. Azizpour, "Explainability techniques for graph convolutional networks," *arXiv preprint arXiv:1905.13686*, 2019.

[56] P. E. Pope, S. Kolouri, M. Rostami, C. E. Martin, and H. Hoffmann, "Explainability methods for graph convolutional neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 10 772–10 781.

[57] T. Funke, M. Khosla, and A. Anand, "Hard masking for explaining graph neural networks," 2020.

[58] M. S. Schlichtkrull, N. De Cao, and I. Titov, "Interpreting graph neural networks for nlp with differentiable edge masking," *arXiv preprint arXiv:2010.00577*, 2020.

[59] X. Wang, Y. Wu, A. Zhang, X. He, and T.-s. Chua, "Causal screening to interpret graph neural networks," 2020.

[60] R. Schwarzenberg, M. Hübner, D. Harbecke, C. Alt, and L. Hennig, "Layerwise relevance visualization in convolutional text graph classifiers," *arXiv preprint arXiv:1909.10911*, 2019.

[61] Q. Huang, M. Yamada, Y. Tian, D. Singh, and Y. Chang, "Graphlime: Local interpretable model explanations for graph neural networks," *IEEE Transactions on Knowledge and Data Engineering*, 2022.

[62] M. Vu and M. T. Thai, "Pgm-explainer: Probabilistic graphical model explanations for graph neural networks," *Advances in neural information processing systems*, vol. 33, pp. 12 225–12 235, 2020.

[63] Y. Zhang, D. Defazio, and A. Ramesh, "Relex: A model-agnostic relational model explainer," in *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*, 2021, pp. 1042–1049.



Yuan Jiang is an assistant professor at the School of Computer Science and Technology, Harbin Institute of Technology. His main research interests include pre-trained code language models, mining software repositories, software vulnerability detection, and source code representation.



Yujian Zhang is a graduate student at the School of Computer Science and Technology, Harbin Institute of Technology. His main research interests include software vulnerability detection and pre-trained code language models.



Xiaohong Su is a professor at the School of Computer Science and Technology, Harbin Institute of Technology. Her research interests include Intelligent software engineering, software vulnerability identification, code representation learning, bug triaging and localization, clone detection, and code search.



Christoph Treude is an Associate Professor of Computer Science at Singapore Management University. His notable achievements include receiving an ARC Discovery Early Career Research Award (2018-2020) and securing funding from industry giants such as Google and Facebook. Treude has been honored with four best paper awards, including two ACM SIGSOFT Distinguished Paper Awards. Currently, he serves on the Editorial Boards of the IEEE Transactions on Software Engineering and the Springer journal on Empirical Software Engineering. Additionally, he is the Open Science Editor for the Elsevier Journal of Systems and Software and has chaired conferences such as ICSME 2020, ICPC 2023, and TechDebt 2023.



Tiantian Wang born in 1980. Received the Doctor's degree from Harbin Institute of Technology, Harbin, Heilongjiang, China, in 2009. Since 2013, she has been an Associate Professor in computer science department of Harbin Institute of Technology. Her current research interests are software engineering, program analysis and computer aided education.